

Table 1: End-to-end execution time (s, median per iteration) and speedup split by pass type. ADT fields = non-recursive fields annotated with @BENCH adt_fields; SoA bufs = 1+ non-recursive field slots across all constructors (recursive children are stored in the tag buffer). Speedup >1× means SoA is faster; **bold** marks >1.1×.

Program	ADT fields	SoA bufs	Fold passes			Map passes		
			AoS (s)	SoA (s)	Speedup	AoS (s)	SoA (s)	Speedup
Compiler	9	8	0.1460	0.0537	2.72 ×	0.1672	0.1807	0.93×
DBQuery	15	13	0.1312	0.0929	1.41 ×	0.1420	0.1728	0.82×
DomTree	15	14	0.1383	0.0640	2.16 ×	0.0590	0.0857	0.69×
KDTree	17	16	0.1913	0.1432	1.34 ×	–	–	–
LinkedListReduction	11	11	0.0309	0.0051	6.01 ×	–	–	–
List	2	2	0.3261	0.3104	1.05×	0.2576	0.3452	0.75×
ObjectGraph	5	4	0.0938	0.0831	1.13 ×	0.1464	0.1674	0.87×
OctTree_sumMass	16	9	0.0356	0.0275	1.30 ×	–	–	–
OctTree_sumEnergy	16	9	0.0357	0.0322	1.11 ×	–	–	–
OctTree_countActive	16	9	0.0350	0.0295	1.19 ×	–	–	–
OctTree_countParticles	16	9	0.0356	0.0280	1.27 ×	–	–	–
OctTree_barnesHutPotential	16	9	0.0420	0.0402	1.04×	–	–	–
OctTree_fmmPotential	16	9	0.0970	0.0976	0.99×	–	–	–
OctTree_scaleEnergy	16	9	–	–	–	0.1517	0.1827	0.83×
OctTree_clearFlags	16	9	–	–	–	0.2733	0.1825	1.50 ×
PiecewiseFunctions	8	7	0.1425	0.0919	1.55 ×	0.2373	0.2670	0.89×
TernaryTree	5	3	0.0300	0.0285	1.05×	0.0861	0.0803	1.07×
Trie	10	9	0.0642	0.0330	1.95 ×	0.1494	0.1721	0.87×
ColorOctree	25	18	0.0908	0.0772	1.18 ×	–	–	–

Table 2: Per-pass performance for `OctTree`, OctTree SoA uses 9 buffers; ColorOctree SoA uses 18 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
barnesHutPotential	F	11/16	31%	0.0420	0.0402	1.04×
clearFlags	M	15/16	6%	0.2733	0.1825	1.50 ×
countActive	F	10/16	38%	0.0350	0.0295	1.19 ×
countParticles	F	8/16	50%	0.0356	0.0280	1.27 ×
fmmPotential	F	12/16	25%	0.0970	0.0976	0.99×
scaleEnergy	M	16/16	0%	0.1517	0.1827	0.83×
sumEnergy	F	12/16	25%	0.0357	0.0322	1.11 ×
sumMass	F	10/16	38%	0.0356	0.0275	1.30 ×
paletteEntriesQuantized	F	13/25	48%	0.0438	0.0343	1.28 ×
quantizationErrorProxy	F	10/25	60%	0.0470	0.0429	1.10×
Total				0.7968	0.6975	1.14 ×
Geomean						1.15 ×

Table 3: Per-pass performance for `Compiler`, ADT has 9 fields, SoA uses 8 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
blockCountPass	F	2/9	78%	0.0130	0.0020	6.61 ×
branchStatsPass	F	2/9	78%	0.0124	0.0035	3.59 ×
castInstCountPass	F	3/9	67%	0.0188	0.0014	13.84 ×
has cycle	F	4/9	56%	0.0364	0.0316	1.15 ×
instCountPass	F	2/9	78%	0.0127	0.0020	6.29 ×
latencyModelPass	F	3/9	67%	0.0133	0.0030	4.46 ×
memoryOpStatsPass	F	3/9	67%	0.0123	0.0038	3.21 ×
stripSideEffectsPass	M	7/9	22%	0.0839	0.0907	0.93×
targetReturnPass	M	9/9	0%	0.0833	0.0900	0.93×
throughputModelPass	F	3/9	67%	0.0137	0.0030	4.57 ×
verifyIR	F	9/9	0%	0.0136	0.0035	3.86 ×
Total				0.3132	0.2343	1.34 ×
Geomean						3.31 ×

Table 4: Per-pass performance for `DBQuery`, ADT has 15 fields, SoA uses 13 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
clearQueryFlags	M	14/15	7%	0.0713	0.0860	0.83×
countJoins	F	3/15	80%	0.0206	0.0118	1.75 ×
filterSelectivitySkew	F	5/15	67%	0.0212	0.0128	1.66 ×
hashJoinPressure	F	5/15	67%	0.0214	0.0127	1.69 ×
scaleCosts	M	15/15	0%	0.0707	0.0868	0.81×
sumCost	F	6/15	60%	0.0201	0.0168	1.20 ×
sumMemory	F	6/15	60%	0.0202	0.0153	1.32 ×
sumRows	F	6/15	60%	0.0276	0.0235	1.17 ×
Total				0.2731	0.2657	1.03×
Geomean						1.25 ×

Table 5: Per-pass performance for `DomTree`, ADT has 15 fields, SoA uses 14 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
SumArea	F	6/15	60%	0.0356	0.0199	1.78 ×
computeWidths	M	14/15	7%	0.0298	0.0481	0.62×
count styled	F	3/15	80%	0.0343	0.0119	2.88 ×
find max Bottom	F	5/15	67%	0.0343	0.0201	1.71 ×
scaleLayout	M	15/15	0%	0.0292	0.0376	0.78×
sumTextWidth	F	3/15	80%	0.0342	0.0121	2.83 ×
Total				0.1973	0.1497	1.32 ×
Geomean						1.51 ×

Table 6: Per-pass performance for `KDTree`, ADT has 17 fields, SoA uses 16 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across $-iterate$ runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
Find nearest Neighbour	F	13/17	24%	0.0298	0.0217	1.37 $\times$
countInRange tight_box	F	13/17	24%	0.0299	0.0205	1.46 $\times$
photonMappingBenchmark	F	12/17	29%	0.0445	0.0423	1.05 $\times$
pointCloudNeighborhood	F	11/17	35%	0.0269	0.0169	1.59 $\times$
sumMassInRange	F	14/17	18%	0.0294	0.0203	1.45 $\times$
twoPointCorrelation bin_8_16	F	11/17	35%	0.0309	0.0215	1.43 $\times$
Total				0.1913	0.1432	1.34 $\times$
Geomean						1.38 $\times$

Table 7: Per-pass performance for `LinearListReduction`, ADT has 11 fields, SoA uses 11 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across $-iterate$ runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
reduction	F	2/11	82%	0.0309	0.0051	6.01 $\times$
Total				0.0309	0.0051	6.01 $\times$
Geomean						6.01 $\times$

Table 8: Per-pass performance for `List`, ADT has 2 fields, SoA uses 2 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across $-iterate$ runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
add1 List	M	2/2	0%	0.2576	0.3452	0.75 $\times$
length List	F	1/2	50%	0.0456	0.0273	1.67 $\times$
sumList	F	2/2	0%	0.1413	0.1425	0.99 $\times$
sumList tail recursive	F	2/2	0%	0.1391	0.1407	0.99 $\times$
Total				0.5837	0.6556	0.89 $\times$
Geomean						1.05 $\times$

Table 9: Per-pass performance for `ObjectGraph`, ADT has 5 fields, SoA uses 4 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across $-iterate$ runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
countLargeItems	F	3/5	40%	0.0133	0.0111	1.20 $\times$
countMarked	F	3/5	40%	0.0132	0.0112	1.18 $\times$
countSurvivors	F	4/5	20%	0.0140	0.0133	1.05 $\times$
deadBytes	F	4/5	20%	0.0137	0.0128	1.07 $\times$
liveBytes	F	4/5	20%	0.0138	0.0129	1.07 $\times$
sumObjIds	F	3/5	40%	0.0125	0.0111	1.13 $\times$
sweepUnmarked	M	5/5	0%	0.0721	0.0832	0.87 $\times$
totalHeapSize	F	3/5	40%	0.0133	0.0108	1.23 $\times$
touchHotObjects	M	5/5	0%	0.0743	0.0842	0.88 $\times$
Total				0.2401	0.2504	0.96 $\times$
Geomean						1.07 $\times$

Table 10: Per-pass performance for `PiecewiseFunctions`, ADT has 8 fields, SoA uses 7 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across $-iterate$ runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
addConstPW	M	8/8	0%	0.1182	0.1354	0.87 $\times$
autorefineMaxLevel	F	4/8	50%	0.0220	0.0193	1.14 $\times$
compressMass	F	3/8	62%	0.0200	0.0114	1.75 $\times$
diffPW	M	8/8	0%	0.1191	0.1316	0.91 $\times$
lbDeuxLoadProxy	F	5/8	38%	0.0220	0.0189	1.16 $\times$
norm2Estimate	F	5/8	38%	0.0256	0.0180	1.42 $\times$
pmapCutHistogram	F	4/8	50%	0.0325	0.0127	2.56 $\times$
truncateTolViolations	F	3/8	62%	0.0205	0.0117	1.76 $\times$
Total				0.3798	0.3589	1.06 $\times$
Geomean						1.36 $\times$

Table 11: Per-pass performance for `TernaryTree`, ADT has 5 fields, SoA uses 3 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across $-iterate$ runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
add 1 tree	M	5/5	0%	0.0861	0.0803	1.07 $\times$
sum tree	F	5/5	0%	0.0300	0.0285	1.05 $\times$
Total				0.1161	0.1088	1.07 $\times$
Geomean						1.06 $\times$

Table 12: Per-pass performance for `Trie`, ADT has 10 fields, SoA uses 9 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across $-iterate$ runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
autocompleteTopKProxy	F	5/10	50%	0.0149	0.0086	1.74 $\times$
countLazyNodes	F	4/10	60%	0.0129	0.0067	1.91 $\times$
countTerminals	F	3/10	70%	0.0121	0.0060	2.02 $\times$
decayTrieStats	M	10/10	0%	0.0744	0.0878	0.85 $\times$
resetTraversalState	M	10/10	0%	0.0750	0.0842	0.89 $\times$
sumPrefixFreq	F	3/10	70%	0.0121	0.0058	2.09 $\times$
sumSubtreeHints	F	3/10	70%	0.0123	0.0059	2.08 $\times$
Total				0.2136	0.2050	1.04 $\times$
Geomean						1.55 $\times$